

Ziplofy Backend – Tech Overview

Ziplofy3b | February 10, 2025 | Express + TypeScript + MongoDB

1. Tech Stack

Core

Layer	Tech	Version
Runtime	Node.js	—
Language	TypeScript	5.3
Framework	Express	4.18
Database	MongoDB	Mongoose 8.18
Auth	JWT + bcrypt	jsonwebtoken 9.x
File upload	Multer	2.x
Email	Nodemailer	7.x

Additional Services

Service	Tech	Purpose
Real-time	Socket.IO	4.8 – live updates, notifications
Job queue	BullMQ + Redis (ioredis)	Async email sending
Archiving	archiver	ZIP creation
Extraction	extract-zip	Theme ZIP extraction
IDs	uuid	Unique identifiers

2. Architecture

Structure

- ~**376 source files** under `Ziplofy3b/src/`
- ~**80+ API routes** (stores, auth, themes, products, orders, customers, discounts, etc.)
- **MVC-style**: Controllers, models, routes, middleware

Request Flow

```
HTTP Request → CORS → express.json() → Route → Auth middleware →  
Controller → MongoDB / File system → Response
```

Auth

- **Admin**: JWT + optional 2FA (OTP via email)
- **Storefront**: Separate storefront auth
- **Middleware**: protect, authorize, permission-based

Database

- MongoDB via Mongoose
- Retry logic (5 retries, 5s delay)
- Env validation at startup

Config

- `.env.development` / `.env.production` via `DOTENV_CONFIG_PATH`
- Required: `PORT`, `MONGODB_URI`, `JWT/ACCESS_TOKEN_SECRET`, `EMAIL_PASSWORD`, `EMAIL_ADDRESS`
- CORS: `localhost` in dev; `*.ziplofy.com + admin.ziplofy.com` in prod

3. What's Working Well

- **Modular layout** – clear separation of routes, controllers, models
- **Async handling** – `asyncErrorHandler` wraps async controllers
- **Error middleware** – central handling (`CustomError`, Mongoose errors)
- **Env validation** – startup fails fast if required vars missing
- **DB resilience** – retry logic for MongoDB connection

- **Graceful shutdown** – SIGTERM/SIGINT closes queue and HTTP server
- **Static files** – /uploads for themes and assets
- **Health endpoint** – / and /api/health for uptime checks

4. Potential Improvements

- **Debug logs** – console.log in auth and controllers; use structured logging and env-based levels
- **Rate limiting** – none on auth/OTP endpoints
- **Helmet** – HTTP security headers not configured
- **Input validation** – no library (Joi/Zod/express-validator); validation is ad hoc
- **@ts-nocheck** – present in index.ts and some controllers
- **Dual bcrypt** – both bcrypt and bcryptjs in package.json; could standardize

5. Summary

The backend is a feature-rich Express + TypeScript API backed by MongoDB, with Socket.IO for real-time updates and BullMQ for background jobs. Architecture and resilience features (env validation, DB retries, graceful shutdown) are solid. The main gaps are around security hardening (rate limiting, Helmet, input validation) and cleaning up debug logging in production.